Assignment 2 Report

Author: By Matthew Favela

Task 1: Constant Brush Implementation

Methods and Implementation

To complete the task of implementing the first "constant" brush which applies a constant color filter in a radius from a mouse point while clicking. To approach this problem, I needed to start by thinking of how to capture a circle around where I click. Then iterate through the area of the circle and change the pixel values to the constant color mask.

My approach to this problem was to:

1. Creating the brush mask by first defining the center of the circle as the brushes radius (half the circle).
2. Then looping through the 2D masks values to see which are inside the area of the circle. If the pixel is in the circle, then I intensified it. I finally flattened this to a 1D array which is my "mask" and returned the mask as the 1D array version

Sub Problem/Implementation: Applying the mask using the brush.

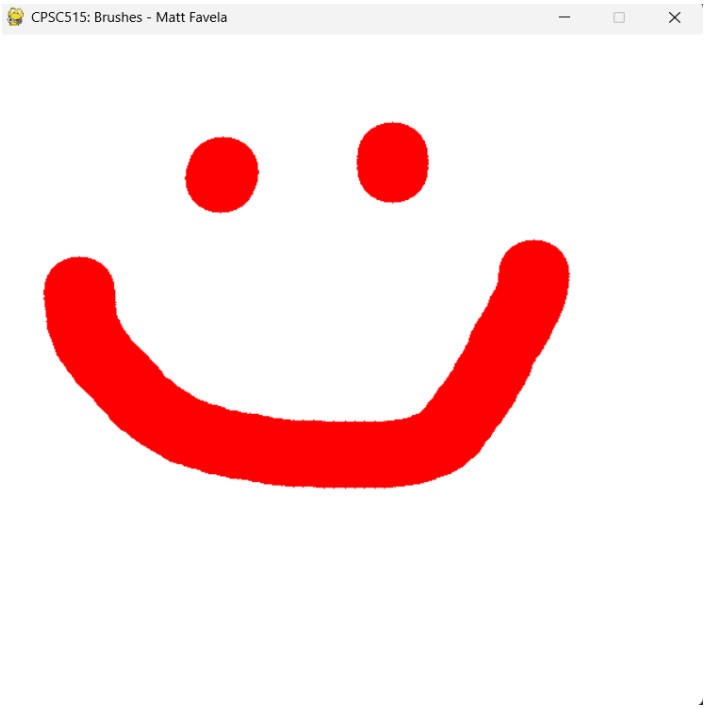
The next method that I needed to implement was the apply brush method. This would be used to loop through the area around my mouse cursor on the canvas (while im holding click), and apply the constant mask over that area. The mask area, and area around the mouse when I click (brush size) will need to be the same width and height, so that I can loop over the brush size and apply the corresponding color from the 1D mask.

My approach to this was to:

1. First confirm the mouse coordinate is in the canvas
2. Loop through the matrix around the mouse click (brush size x brush size)
3. Calculate the x and y offset, to set the origin to the bottom left of the matrix. This was done by simply taking the mouse x position and subtracting the brushes radius, then adding the index of which column and row we're at going through the matrix, based on the current index of the for loops.
4. Next, i put another check if the pixel were about to alter is actually inside the canvas, as a safety precaution. If this passes then we move on
5. There is a few things I will need to keep track of, 5a. The mask index which is our current index position in the *flattened* 1D mask array 5b. The canvas index which will keep track of which position in the *data* 1D array, we're at.
6. From here I can refernce the intensity of pixel were at in the mask, and the intensity of the pixel at its current state (converted from uint8 to float format for calculations)
7. Next I prepared the the actual brush color's intensity that we will apply to the pixel. This is (255, 0, 0, 255) meaning a red and fully transparent color, converted to a 0-1 scale for its intensity in each RGBA slot.
8. To do color mixing, I needed to create a value that added up to 0-1 intensity, so I started by adding the mask intensity and the current pixel intensity.

9. Finally I needed to convert the 0-1 value to a uint8 value, so I multiplied it by 255.

With these componenets setup, I am now able to test and visualize the first brush.



Task 2: Linear Brush Implementation

Methods and Implementation

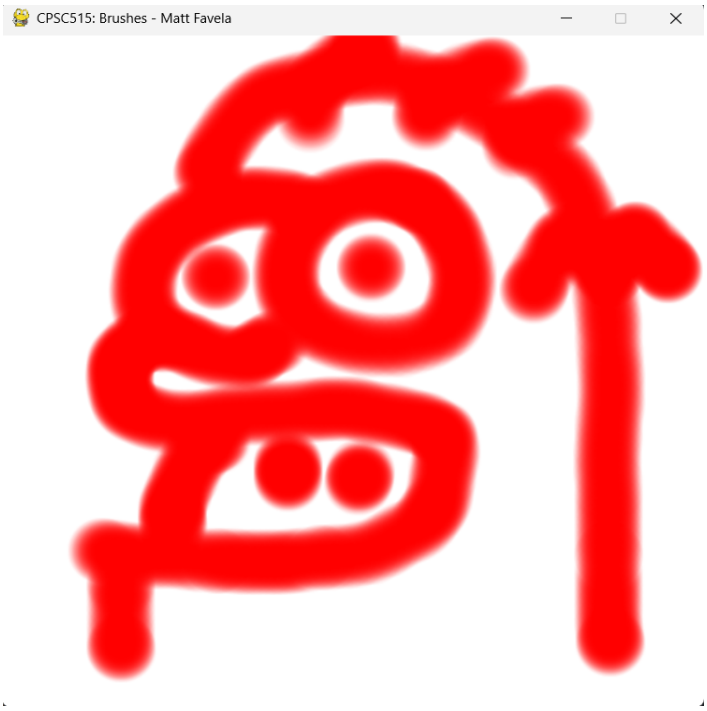
To complete thhe task of implementing a linear brush, I needed to:

1. Understand linear distribution
2. Set up the mask formula
3. Implement color mixing

My approach to this was problem was to:

1. Start by understanding linear distribution, and how a linear brush would be implemented using a mask. I needed to essentially use the distance, to subtract from the max intensity so that the intensity decreases the further away it is.
2. Next I needed to set up the mask formula, which is just the distance from the center point, subtracted from the max intensity.
3. I used the formula to do the same looping as before, but with different intensity values from the distance

Overall, I think that after setting up the apply brush method, it was pretty easy to test and prototype the new brush.



Task 3: Quadratic Brush Implementation

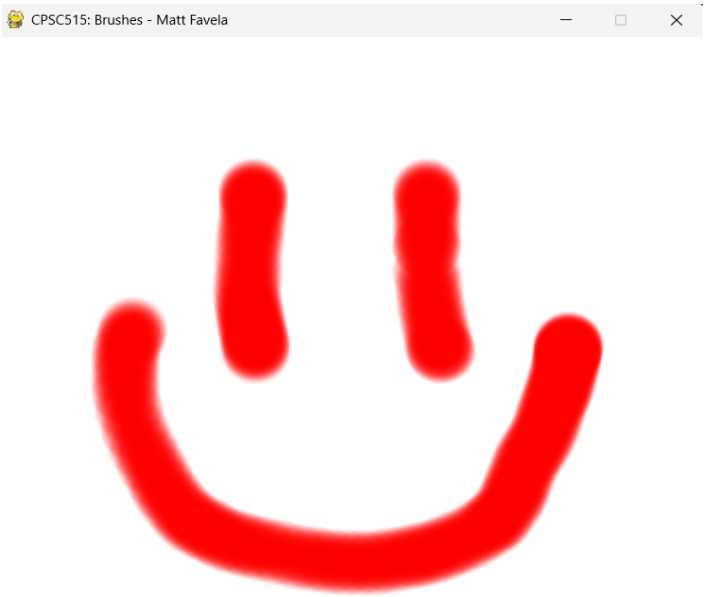
Methods and Implementation

To accomplish this task, I would need to:

1. I started by setting up my coefficients for the quadratic equation. A is the negative coefficient, B is the 0 coefficient, C is the positive coefficient. (f(d) = A*d^2* + *B*d + C)
2. Next, I setup my loop for the mask and started by calculating the distance from the center point to the current pixel in iteration. If this distance is less than the radius of the brush.
3. Finally I would use my quadratic equation to calculate the intensity of the pixel, with 0 being the minimum.

With this, I was able to return the 1D flattened array mask and tested it on the canvas.

Results



Task 4: Extra Brush Implementation - Spray Paint

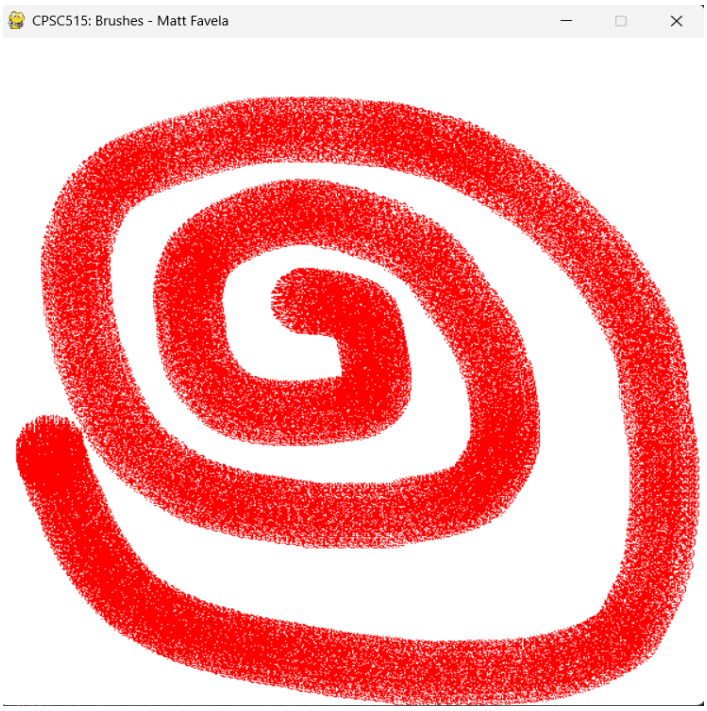
Methods and Implementation

To complete this task I needed to create a new parameter for the brushes density, then use it as the density of the brush. This would create a spray paint brush effect.

My goal was to:

1. Creating the randomized mask by looping through the 2D mask and setting the value at each index to a random number between 0 and 1. To do this, I imported the random library.
2. Setting up density parameters by taking in a density value when initializing your brush. This is what is used in the spray pain brush mask.
3. Set the pixel intensity to the density value, if the random value was less than the defined density. This means 25% of the time we will set the pixel intensity to the density value.

Results



Task 5: Extra Brush Implementation - Speed Brush

Methods and Implementation

To complete this task, I needed to implement a brush that would start off as a smaller size, and grow the faster you move your mouse.

My approach to this was to:

1. Start by calculating the distance from the current pixel to the center pixel.
2. Use this distance to scale the intensity, then use a higher exponent to quickly increase the intensity over time.
3. Apply this to the 2D mask, flatten, and return it.

Results

CPSC515: Brushes - Matt Favela